

Data Structures; an Introduction

The computer as a machine can be used for among many things:

- ❖ Scientific Calculations
- ❖ Information Management System for an enterprise
- ❖ Real-time control system for an assembly line

Introduction

- In performing these operations, to use the computer effectively one needs to:
 - a) Obtain the needed data and their relations
 - b) Store the data into the computer according to their relations
 - c) Design algorithms to solve the problem

Introduction

- The essence of programming and data processing requires *efficient algorithms* for accessing the data both in main memory and on secondary storage devices.
- This efficiency is directly linked to the structure of the data being processed. A data item that can be effectively linked to other data items takes on meaning that transcends its individual contents.
- A data structure is a way of organizing data that considers not only the items stored but also their relationship to each other.

Introduction

- The way data is accessed is a function of how it is organized, thus a general understanding of data structures is essential to developing efficient algorithms.
- A clear notion of the relative advantages and disadvantages of each technique is obviously crucial to those designing systems.

Introduction

- The ability to make correct decisions regarding which data structures to use typically involve the following general issues:
 - a) The efficiency of the program with respect to its **run time**. Does it perform the optimal number of operations to achieve its goal?
 - b) The efficiency of a program with respect to **its utilization of main memory and secondary storage** devices. Does it consume such resources in a fashion that makes its use impractical?
 - c) The **developmental costs** of a program or system. Could a different approach to the problem significantly reduce the total person-hours invested in it?

Introduction

- Thorough knowledge of a programming language is not a sufficient base upon which to make these decisions.
- The study of data structures will expose you to a vast collection of tried and proven methods used in designing efficient programs.
- In many applications, different types of data structures could be used to achieve the same end. In such situations, the designer plays a game of **tradeoffs**.
- One data structure sacrifices memory compactness for speed; another utilizes memory efficiently but results in a slow run time.
- For each positive there is seemingly a corresponding negative.

Introduction

- No absolute best data structure exists. Instead, designers must rely on their knowledge of the strong and weak points of various data structures to choose the one that best fits each application.

There are several kinds of Data Structures:

- ***Mathematical data structures*** – with mathematical operations such as addition, subtraction, multiplication and division.
- ***Lists, Trees or graphs*** – with non-numerical operations e.g. searching, insertions, deletion of elements in specific areas

Algorithms for Data structures

- Once a data structure has been chosen for a particular application, it is given life by the **logical instructions** that manipulate the related data items stored in it i.e. the algorithm.

Desirable traits of an algorithm are (among others):

- i. completely free of ambiguity.
- ii. It must be **efficient**. It should not unnecessarily use memory locations nor should it require an excessive number of logical operations.
- iii. Should be **concise and compact** to facilitate verification of their correctness.

Algorithms for Data structures

- **Verification** involves observing the performance of the algorithm with a carefully selected set of test cases.
- These test cases should attempt to cover all of the exceptional circumstances likely to be encountered by the algorithm. These exceptional circumstances will depend upon the data structure being manipulated.
- However, certain generic circumstances must be verified for all algorithms manipulating data structures:
 - a) Does the algorithm work when the data structure is empty?
 - b) Does the algorithm work when the data structure is full?
 - c) Does the algorithm work for all the possibilities that can occur between an empty structure and a full structure?

Algorithms for Data structures

- The need for **conciseness** in algorithms becomes obvious when you consider the problem of verifying their correctness. Hence, an algorithm should concern itself with one specific problem.
- Each algorithm is a logical module designed to handle a specific problem relative to a particular data structure.
- Such modules could be tied together by higher level algorithms, which focus upon the connections between modules rather than the underlying data structures.

Good Programs

There are a number of facets to good programs: they must

- a) run correctly
- b) run efficiently
- c) be easy to read and understand
- d) be easy to debug
- e) be easy to modify

Basic *definitions*

- i. **Data structures** - In computer science, this is a way of storing data in a computer so that it can be used efficiently. Often a carefully chosen data structure will allow a more efficient algorithm to be used.
- ii. An **abstract data type (ADT)** - is a specification of a data structure and the set of operations that can be performed on it. E.g a queue
- iii. **Abstraction** - This is the separation between what a data structure represents and what an algorithm accomplishes, from the implementation details of how things are actually carried out. i.e., hiding the unnecessary details
- iv. **Data Abstraction** - Hiding of the representational details
- v. **Data Types** - **a data type**, or **type** is a classification of data. It indicates a set of values that have the same general meaning or intended purpose, e.g primitive types(integers,strings),records,classes

Atomic/Simple Data Types

- These are data types that are defined without imposing any structure on their values (have no internal structural relationship)
- Examples of atomic types.
 - Boolean,
 - Integer
 - characters

Structured Data Types

- A structured data type has a definition that imposes structure upon its values.
- In many structured data types, there is an *internal structural relationship*, or organization, that holds between the components.
- For example, if we think of an array as a structured type, with each position in the array being a component, then there is a structural relationship of '*followed by*': we say that component i is followed by component $i+1$.

Structural Relationships

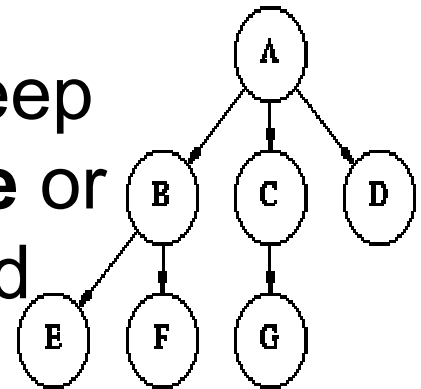
- Many structured data types do have an internal structural relationship, and these can be classified according to the properties of this relationship.

Structured Data types: Linear Structure

- The most common organization for components is a linear structure. A structure is linear if it has these 2 properties:
 - Property P1: Each element is 'followed by' at most one other element.
 - Property P2: No two elements are 'followed by' the same element.
- An array is an example of a linearly structured data type. We generally write a linearly structured data type like this: $A \rightarrow B \rightarrow C \rightarrow D$ (this is one value with 4 parts).
- Counter example 1 (violates P1): A points to B and C $B \leftarrow A \rightarrow C$
- Counter example 2 (violates P2): A and B both point to C
 $A \rightarrow C \leftarrow B$

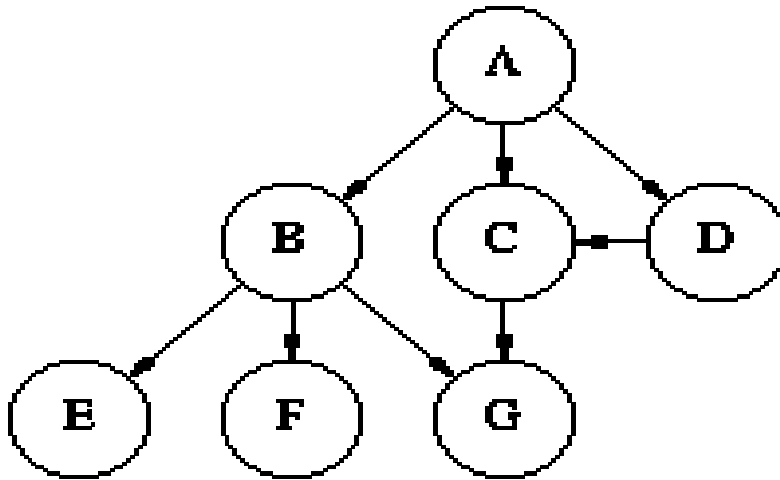
Structured Data types: Classes: **Tree Structure**

- If we drop the first constraint (**P1**) and keep the second (**P2**) we get a **tree structure** or **hierarchy**: no two elements are followed by the same element. This is a very common structure too, and extremely useful.



- Counter example 1 in the slide before is a tree, but counter example 2 is not.
- A is followed by B C D, B by E F, C by G.

Structured Data types: Classes: **Graph Structure**

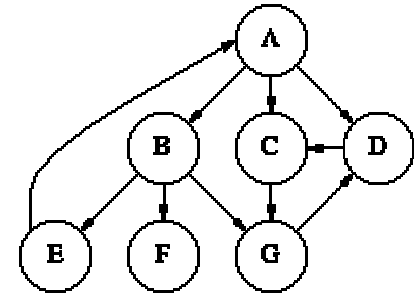


Dropping both P1 and P2

- If we drop both constraints, we get a **graph**.

Structured Data types: Classes: **Cyclic Structures:**

- All the examples we've seen are **acyclic**. This means that there is no sequence of arrows that leads back to where it started.
- Linear structures are usually acyclic, but cyclic ones are not uncommon.

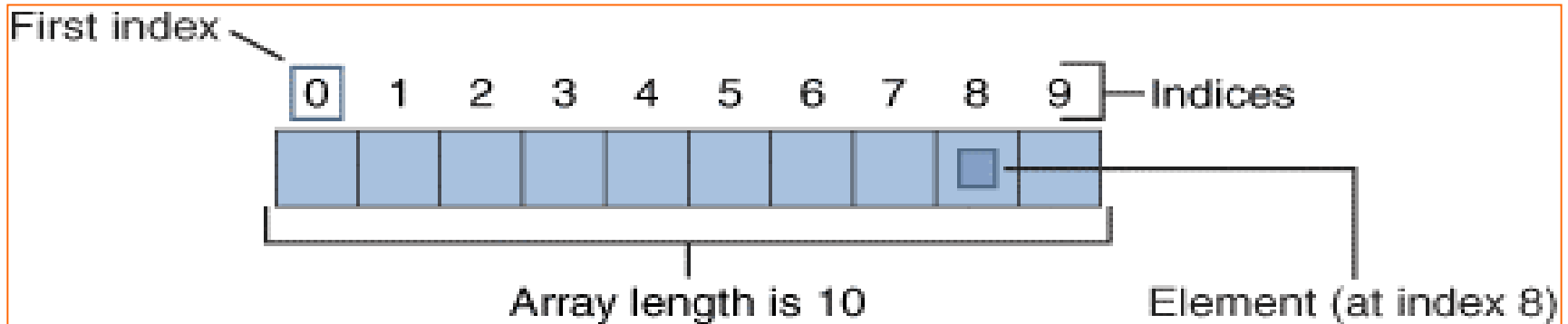


- Graphs are often cyclic, although acyclic graphs are an important topic of study.
- Example: Add an edge from G to D, and from E to A.

Structured Data Types : Arrays

- An Arrays is a **physically sequential**, fixed size collection of homogeneous objects. In addition, objects with the structure have the random access property.
 - I. The array is physically sequential in that the data objects are stored in consecutive memory locations.
 - II. The array has a fixed size in that its size can neither be increased nor reduced though the number of items it contains can vary
 - III. The array is homogeneous in that they are made up of objects that are all the same type.
 - IV. The random access property means that the time it takes to access one object in the structure does not depend on what object in the structure had been accessed previously.

Arrays



- An array type is appropriate for representing an abstract data type when the following three conditions are satisfied:
 - ❖ The data objects in the abstract data type are composed of homogeneous objects
 - ❖ The solution requires the representation of a fixed, predetermined number of objects
 - ❖ There is an ordering that can be placed on the objects in the abstract data type

Arrays

```
int Numbers[5];
```

0	1	2	3	4

```
for (int i=0;i<5;i++)  
{  
    printf("Enter Next Number:-  
");  
    scanf("%d",&num);  
    printf("\n\n");  
    Numbers[i]=num;  
}
```

8	2	22	56	11
---	---	----	----	----

```
int big=Numbers[0];  
for (i=0;i<5;i++)  
{  
    if(Numbers[i]>big)  
        big=Numbers[i];  
}
```

Big=8	Big=8	Big=22	Big=56	Big=56
-------	-------	--------	--------	--------

Arrays

// arrays example

```
#include <stdio.h>
int billy [] = {16, 2, 77, 40, 12071};
int n, result=0;
int main ()
{
    for ( n=0 ; n<5 ; n++ )
    {
        result += billy[n];
    }
    printf("%d",result);
    return 0;
}
```

Arrays

```
1. #include <stdio.h>
2. int Numbers[5];
3. void main(void)
4. {
5.     printf("We are dealing with arrays \n");
6.     printf("**** Assign values ***\n");
7.     printf("\n\n");    int num=0;
8.     for (int i=0;i<5;i++)
9.     {    printf("Enter Next Number:- ");
10.         scanf("%d",&num);
11.         printf("\n\n");
12.         Numbers[i]=num;
13.     }
14.     // Print the elements
15.     printf("Printing elements \n");
16.     printf("\n\n");
17.     for ( i=0;i<5;i++)
18.     {
19.         printf("%d",Numbers[i]);
20.         if(i<4) printf(" : ");
21.     }
```

```
22.         //Get the biggest elemnt
23.         printf("\n\n");
24.         int big=Numbers[0];
25.
26.         for (i=0;i<5;i++)
27.         {
28.             if(Numbers[i]>big)
29.                 big=Numbers[i];
30.         }
31.         printf("The Biggest number is
32.         %d", big);
33.         printf("\n\n");
34.     }
```

Arrays

CA "C:\MSC_COURSES\CHEPKEN\Arrays\Debug\Arrays.exe"

```
*** We are dealing with arrays ***  
*** Assign values ***
```

```
Enter Next Number:- 8
```

```
Enter Next Number:- 2
```

```
Enter Next Number:- 22
```

```
Enter Next Number:- 56
```

```
Enter Next Number:- 11
```

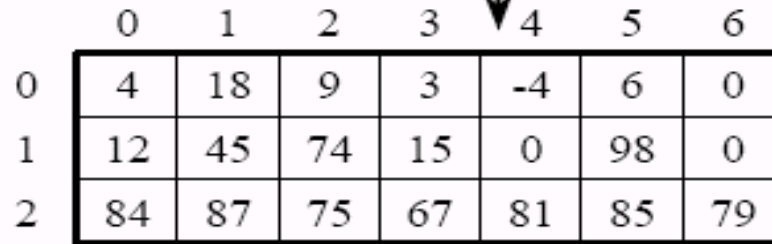
```
Printing elements
```

```
8      : 2      : 22     : 56     : 11
```

```
The Biggest number is 56
```

B Multidimensional arrays

```
const int NUMROWS = 3;  
const int NUMCOLS = 7;  
int Array[NUMROWS][NUMCOLS];
```



	0	1	2	3	4	5	6
0	4	18	9	3	-4	6	0
1	12	45	74	15	0	98	0
2	84	87	75	67	81	85	79

Array[2][5] 3rd value in 6th column
Array[0][4] 1st value in 5th column

- Multidimensional arrays can be described as arrays of arrays. For example, a two-dimensional array consists of a certain number of rows and columns:
- A one-dimensional array is usually processed via a 'for' loop. Similarly, a two-dimensional array may be processed with a nested for loop:

What does the piece of code below do?

```
for (int Row = 0; Row < NUMROWS; Row++)  
{  
    for (int Col = 0; Col < NUMCOLS; Col++)  
    {  
        Array[Row][Col] = 0;  
    }  
}
```

What would be the print out of the following initialization?

```
int Array1[2][3] = { {1, 2, 3} , {4, 5, 6} };
int Array2[2][3] = { 1, 2, 3, 4, 5 };
int Array3[2][3] = { {1, 2} , {4} };

for (int row = 0; row < 2; row++)
{
    for (int col = 0; col < 3; col++)
    {
        printf("%d",Array1[row][col]);
    }
    printf("\n\n");
}
```

Arrays:Disadvantages

- 1) The size of the array is fixed —
Most often this size is specified at compile time with a simple declaration such as in the example above. With a little extra effort, the size of the array can be deferred until the array is created at runtime, but after that it remains fixed.
- 2) Because of (1), the most convenient thing for programmers to do is to allocate arrays which seem "large enough" (e.g. the 100 items). Although convenient, this strategy has two disadvantages:
 - most of the time there are just 20 or 30 elements in the array and 70% the space in the array really is wasted.
 - If the program ever needs process more than 100 scores, the code breaks.
- 3) Inserting new elements at the front or somewhere at the middle is potentially expensive because existing elements need to be shifted over to make room.

Exercises

- 1) A matrix can be represented using a two dimensional array. Create a program that does the following
 - i. Write a C program that creates a three by three matrix.
 - ii. Allows a user to enter values into the elements of the matrix
 - iii. Outputs the elements of the matrix
 - iv. Sums all the elements of the matrix and outputs the result